

Programación Orientada a Objetos

Pablo González, Víctor Peiró, Gonzalo Martínez

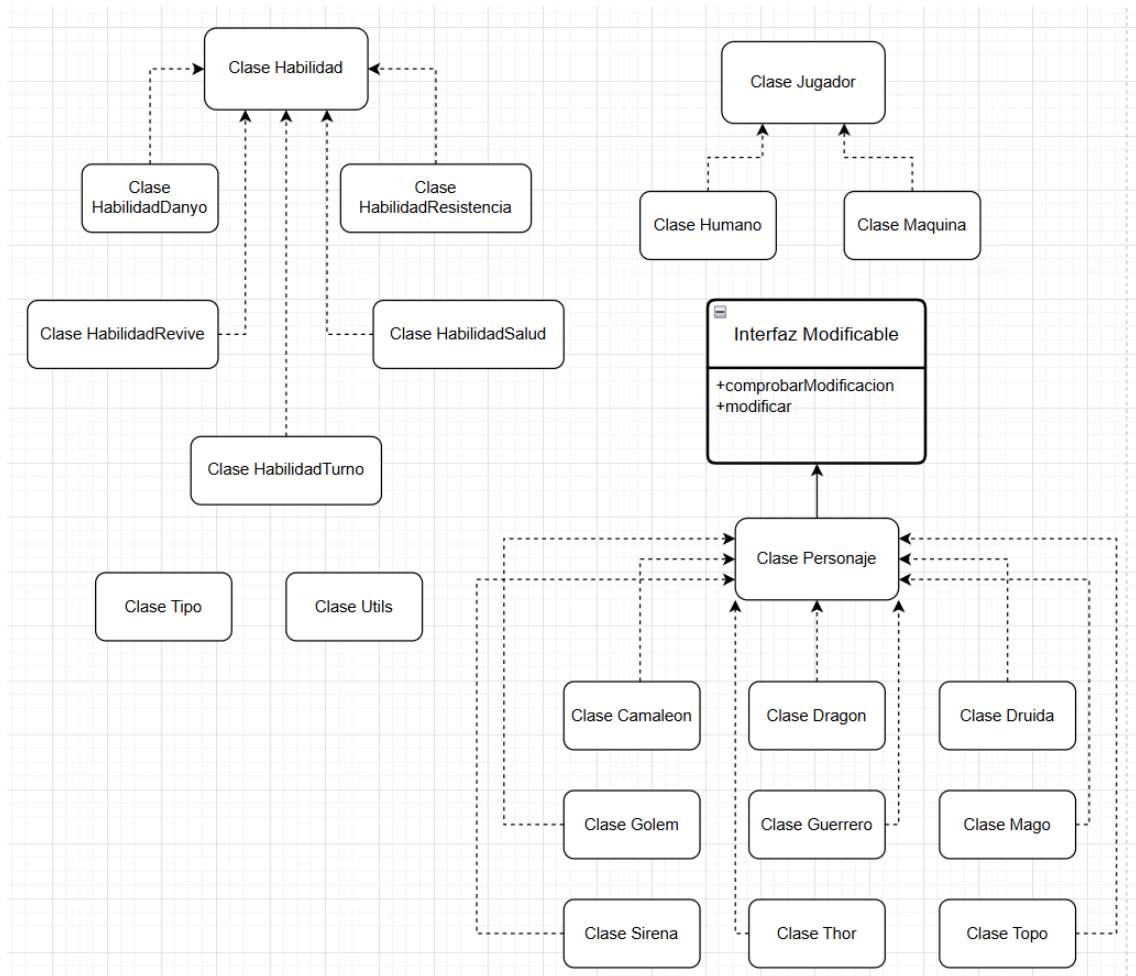
PROYECTO FINAL

Battle Royale

I. Concepto General:

El programa imita un juego de battle royale y mezcla conceptos con juegos por turnos. Al inicio de la ejecución, se pedirá al usuario que introduzca el número de jugadores controlados por humanos y el número de jugadores controlados por el ordenador. Cada jugador tanto personaje como máquina podrá elegir un personaje que tendrá un ataque diferente, hay personajes capaces de cambiar el estado del personaje mientras que hay otros capaces de atacar 2 veces o cambiar estadísticas base de los personajes. Cada personaje empieza con una habilidad que puede consumir. Al consumir una habilidad, el jugador tendrá derecho a elegir una habilidad nueva. La partida acabará una vez solo quede un jugador con vida.

II. Esquema:



III. Jerarquía

Clases Únicas:

- Tipo
- Utils

Hijos de Clase Personaje: Hereda de interfaz Modificable

- Camaleon
- Druida
- Dragon
- Golem
- Guerrero
- Mago
- Sirena
- Thor
- Topo

Hijos de Clase Habilidad:

- HabilidadTurno
- HabilidadDanyo
- HabilidadSalud
- HabilidadResistencia
- HabilidadMaquina

Hijos de Clase Jugador:

- Maquina
- Humano

IV. Interfaz Modificable:

La interfaz `Modificable` define métodos para gestionar cambios en el juego en función de condiciones específicas. El método `comprobar Modificación` verifica si se cumplen los requisitos para realizar una modificación, utilizando una lista de jugadores y el número de rondas como parámetros. El método `modificar` aplica dichos cambios a los jugadores, adaptando sus características o estados según la lógica del juego.

V. Clases:

- Clase Personaje:

La clase `Personaje` implementa la interfaz `Modificable` y representa un personaje con atributos como nombre, salud, daño, resistencia, tipo y habilidad, permitiendo gestionar su comportamiento dentro del juego. Proporciona métodos para realizar ataques, aplicando reglas como ventajas o desventajas entre tipos de personajes. Además, incluye un sistema de evolución que mejora al personaje si se cumplen ciertas condiciones, como tener poca vida y estar en rondas avanzadas, aunque esta mejora no siempre tiene éxito. También dispone de funciones para imprimir detalles del personaje y realizar modificaciones en su estado durante el transcurso del juego.

- **Clase Camaleón:** Cuando ataca, cambia su tipo a el de la persona a la que ataca
- **Clase Dragón:** Ataca a 2 enemigos
- **Clase Druida:** Cuando ataca, se cura 5 de vida
- **Clase Golem:** Al atacar, baja la resistencia del enemigo
- **Clase Guerrero:** Tiene vida y daño aleatorios
- **Clase Mago:** Sus ataques queman al rival, penalizándole en los siguientes turnos. Cambiándole el estado a quemado
- **Clase Sirena:** Cuando le atacan, confunde al rival, habiendo probabilidad de que se auto ataque el rival.
- **Clase Thor:** Ataca a un enemigo, y el ataque rebota por los rivales, bajando por cada rival 1 unidad el daño que hace.
- **Clase Topo:** Tiene posibilidades de fallar el ataque, pero si consigue atacar, hace un gran daño.

- Clase Jugador:

La clase `Jugador` actúa como la base para los participantes del juego, dividiéndose en jugadores controlados por humanos y por la máquina. Define atributos clave como el

nombre, el orden en la ronda, el personaje asociado y si tiene el turno activo. Implementa métodos para gestionar el uso de habilidades (como curar, aumentar daño o revivir), verificar si el personaje ha muerto y manejar comparaciones entre jugadores según su orden en la ronda. Además, contiene métodos abstractos que sus clases hijas deben implementar para definir cómo realizan sus turnos y cómo seleccionan a sus objetivos. También incluye una función simplificada para mostrar información relevante del jugador.

- **Clase Humano:**

La clase `Humano` extiende de `Jugador` y representa a los jugadores controlados por humanos, permitiendo que el usuario tome decisiones sobre las acciones en cada turno. Durante el turno, muestra las opciones disponibles (atacar, usar habilidades, adquirir nuevas habilidades o evolucionar) y valida las selecciones del jugador. También permite elegir manualmente a los objetivos de las acciones, asegurándose de que las decisiones sean coherentes, como no atacarse a sí mismo o seleccionar objetivos válidos. Además, gestiona habilidades específicas como ataques múltiples, efectos en otros jugadores y transformaciones del personaje. La clase mantiene una interacción constante con el usuario a través de mensajes en consola para guiarlo en su estrategia.

- **Clase Maquina:**

La clase `Maquina` extiende de `Jugador` y representa a los jugadores controlados por la inteligencia artificial. En su turno, todas las decisiones se toman de forma aleatoria, como seleccionar un objetivo para atacar, usar habilidades o evolucionar. Si el personaje tiene una habilidad, la IA la usa según el tipo de habilidad (como daño, resistencia o control de turno). También incluye la lógica de ataques especiales, como los que realiza el personaje `Dragon`, que puede atacar a dos jugadores. Si está disponible, también puede evolucionar el personaje o recibir una nueva habilidad de forma aleatoria. Además, tiene un método para escoger aleatoriamente a su objetivo de ataque, asegurándose de no atacarse a sí mismo.

- **Clase Habilidad (Tipo Genérico):**

La clase `Habilidad` es abstracta y sirve como base para definir habilidades que pueden modificar atributos de los personajes y jugadores. Contiene atributos como el nombre de la habilidad, la probabilidad de uso y un multiplicador que afecta cómo se aplican los efectos de la habilidad. Tiene dos constructores, uno que incluye un multiplicador y otro que no, útil para habilidades como revivir que no requieren un multiplicador. Además, proporciona los métodos para obtener y establecer estos atributos, un método `toString` para representar la habilidad como texto y un método abstracto `usarHabilidad`, que debe ser implementado por sus clases hijas para definir cómo cada habilidad afecta a un atributo específico del personaje o jugador.

- **Clase Habilidad Danyo:**

La clase `HabilidadDanyo` es una subclase de `Habilidad` que se especializa en modificar el daño infligido. Esta clase implementa el método `usarHabilidad`, que aumenta o disminuye el daño según un multiplicador, dependiendo de la probabilidad de éxito definida en la habilidad. Si la habilidad tiene éxito, se aplica el multiplicador al daño y se muestra un mensaje indicando si el personaje se ha vuelto más fuerte o si sus golpes han perdido efectividad. Si la habilidad falla, se notifica al jugador. El método `toString` proporciona una representación en texto de la habilidad, incluyendo la información de la clase padre.

- **Clase Habilidad Resistencia:**

La clase `HabilidadResistencia` es una subclase de `Habilidad` que se centra en modificar la resistencia de un personaje. Al igual que `HabilidadDanyo`, implementa el método `usarHabilidad`, que aplica un multiplicador a la resistencia del personaje, dependiendo de la probabilidad de éxito de la habilidad. Si la habilidad tiene éxito, aumenta o disminuye la resistencia según el valor del multiplicador y muestra un mensaje que indica si el personaje se ha vuelto más resistente o débil. Si la habilidad falla, también se informa al jugador. El método `toString` proporciona una representación textual de la habilidad, incluyendo la información de la clase padre.

- **Clase Habilidad Revive:**

La clase `HabilidadRevive` es una subclase de `Habilidad` que se especializa en revivir a un personaje, restaurando una parte de su salud. Esta habilidad no utiliza un multiplicador, por lo que en su método `usarHabilidad`, si tiene éxito, devuelve la mitad del valor de salud proporcionado como argumento. Si la habilidad falla, se muestra un mensaje indicando el fracaso. El método `toString` proporciona una representación textual de la habilidad, incluyendo el nombre y la probabilidad de uso, de acuerdo con la implementación de la clase base.

- **Clase Habilidad Salud:**

La clase `HabilidadSalud` es una subclase de `Habilidad` que se enfoca en curar al personaje. Implementa el método `usarHabilidad`, que restaura una parte de la salud del personaje según el multiplicador de la habilidad. Si la habilidad tiene éxito, se calcula la curación en función de la salud actual y el multiplicador, y se muestra un mensaje indicando cuántos puntos de salud se han restaurado. Si la habilidad falla, se notifica al

jugador. El método `toString` devuelve una representación textual de la habilidad, incluyendo la información de la clase base.

- Clase Habilidad Turno:

La clase `HabilidadTurno` es una subclase de `Habilidad` que permite desactivar el turno de un oponente. Implementa el método `usarHabilidad`, que cambia el estado del turno del oponente a **inactivo** si la habilidad tiene éxito, dependiendo de la probabilidad de uso. Si la habilidad falla, se muestra un mensaje notificando el error. El valor que se pasa al método es un booleano que indica si el turno del oponente está activo o no. El método `toString` devuelve una representación textual de la habilidad, incluyendo el nombre y la probabilidad de uso.

- Clase Tipo:

La clase `Tipo` representa el tipo de un personaje en el juego, definiendo sus fortalezas y debilidades frente a otros tipos. Utiliza la enumeración `Tipos` para asignar un nombre al tipo y dos listas para almacenar sus debilidades y fortalezas (tipos contra los cuales es débil o fuerte). La clase ofrece métodos para verificar si un tipo es débil o fuerte frente a otro, y para comparar tipos en función de estas relaciones. Implementa la interfaz `Comparable<Tipo>`, lo que permite comparar tipos y ordenar objetos de este tipo según si un tipo es más fuerte o débil que otro. También incluye métodos para obtener y establecer sus atributos, y un método `toString` para mostrar los detalles del tipo.

- Clase Utils:

La clase `Utils` tiene 3 funciones útiles que se utilizan varias veces en el juego que son:

1. **`pedirNumero(int numMin, int numMax)`**: Pide un número al usuario dentro de un rango específico. La entrada se valida para asegurarse de que el valor ingresado sea un número entero dentro del rango proporcionado. Si la entrada es inválida (no un número o fuera de rango), se pide nuevamente.
2. **`pedirCadena()`**: Pide una cadena de texto al usuario. Si la entrada es incorrecta, se vuelve a solicitar la cadena hasta obtener una válida.
3. **`generarNumeroAleatorio(Integer num)`**: Genera un número aleatorio entre 0 y el número especificado como parámetro.

Estas funciones ayudan a manejar la entrada del usuario y la generación de números aleatorios de manera eficiente y con validación de errores.

- Enumerados: (Estados/Tipos)

El enumerado de los estados se usa para acceder más fácil a la lista de estados en

los cuales pueden estar los personajes.

El enumerado tipos hace referencia a los distintos tipos que puede ser cada personaje, el juego asigna un tipo a cada personaje, junto a una cualidades.

- Clase Partidas Maxima:

La clase principal PartidasMaximas se encarga de gestionar el desarrollo de una partida en el juego. Su propósito es crear y configurar todos los elementos necesarios para una partida, como los tipos de personajes, habilidades, jugadores y personajes en sí. A continuación, te explico de forma detallada sus funciones:

1. Inicialización de la partida:

- En el constructor, se crean los tipos de personajes, las habilidades disponibles y los personajes iniciales.
- Luego, se solicita al usuario que ingrese el número de jugadores humanos y máquinas, asegurándose de que haya al menos un jugador humano y en total, al menos dos jugadores.
- Los jugadores se asignan a posiciones aleatorias en la ronda.

2. Creación de tipos y personajes:

- La clase genera una lista de tipos de personajes con sus respectivas fortalezas y debilidades.
- También crea una lista de habilidades que los personajes pueden usar durante el juego.
- Los personajes se generan aleatoriamente, asignándoles un tipo y una habilidad.

3. Simulación de rondas:

- En cada ronda, se cicla por los jugadores vivos, donde cada uno realiza su acción dependiendo de su estado (normal, quemado, paralizado, etc.).
- Si un jugador está quemado, pierde salud. Si está paralizado o en algún otro estado que impide su turno, se le intenta liberar de dicho estado.
- Cada jugador puede realizar una acción como atacar, usar una habilidad o evolucionar, dependiendo de las reglas definidas para su personaje.

4. Fin de la partida:

- El juego continúa hasta que solo queda un jugador vivo, y se imprime el ganador junto con los resultados de todos los jugadores eliminados.

5. Métodos de apoyo:

- **asignarPosiciones:** Asigna posiciones aleatorias a los jugadores para determinar el orden en el que participan en cada ronda.
- **crearTipos, crearHabilidades, crearPersonajes:** Métodos que generan los tipos de personajes, las habilidades y los personajes disponibles para los jugadores.
- **jugarRonda y jugarPartida:** Métodos que simulan las rondas del juego y gestionan el avance hasta que se determina un ganador.
- **imprimirResultados:** Muestra al ganador y el estado de los jugadores muertos.

En resumen, esta clase gestiona la creación, el flujo de las rondas y la determinación de un ganador en el juego, integrando todos los aspectos del combate y las interacciones entre personajes.

